

<テキスト資料> 螺旋運動ノードを作ってみよう

my_node.cppの変更

```
#include "rclcpp/rclcpp.hpp"
#include "geometry_msgs/msg/twist.hpp"

class SpiralPublisher : public rclcpp::Node
{
public:
    SpiralPublisher()
    : Node("spiral_publisher"), count_(0)
    {
        publisher_ = this->create_publisher<geometry_msgs::msg::Twist>("/turtle1/cmd_vel", 10);
        timer_ = this->create_wall_timer(
            std::chrono::milliseconds(100),
            std::bind(&SpiralPublisher::timer_callback, this));
    }

private:
    void timer_callback()
    {
        auto msg = geometry_msgs::msg::Twist();
        msg.linear.x = 0.5 + 0.05 * count_; // 前進速度を徐々に増加
        msg.angular.z = 2.0;               // 回転速度は一定
        publisher_>publish(msg);
        RCLCPP_INFO(this->get_logger(),
            "Publishing spiral: linear=%.2f, angular=%.2f",
            msg.linear.x, msg.angular.z);
        count_++;
    }

    rclcpp::Publisher<geometry_msgs::msg::Twist>::SharedPtr publisher_;
    rclcpp::TimerBase::SharedPtr timer_;
    int count_;
};

int main(int argc, char * argv[])
{
    rclcpp::init(argc, argv);
    rclcpp::spin(std::make_shared<SpiralPublisher>());
    rclcpp::shutdown();
    return 0;
}
```

パッケージのビルド

```
cd ~/colcon_ws
colcon build
source install/setup.bash
```

別ターミナルでTurtlesimを起動

```
ros2 run turtlesim turtlesim_node
```

元ターミナルで自作ノードを起動

```
ros2 run my_cpp_pkg my_cpp_node
```